



Ayoub Agouzoul
Georgii Kuznetsov
Georgy Salakhutdinov

September 8, 2025

ZERO-SHOT NUTRITION ESTIMATION

Leveraging Deep Neural Networks for Automated Food Analysis

Abstract

Keeping track of our food intake can still feel like a chore, even with so many nutrition apps available today. This project explores a potential solution through Machine Learning. We frame this challenge as a multi-output regression task. Given an RGB image of a plated dish, we want to predict its total weight together with calories, fat, carbs, and protein. The Nutrition5K dataset from Google Research has been utilized for training. This paper makes 3 contributions. Firstly, we present our data pipeline, which involves cleaning, segmentation, and a Principal Component Analysis (PCA) video frame extraction strategy. Secondly, we present our model WeightCNN that has been trained to predict weight with 18.7% relative MAE. Finally, we present a family of relative-macro models that have been trained to estimate macronutrient percentages. These models are then combined with WeightCNN to obtain the full calorie and macronutrient prediction from a single photo.

Acknowledgements We would like to express our sincere gratitude to Professor Jesse Read and Professor Adrien Ehrhardt, for their invaluable guidance, and encouragement throughout the course of this project.

Contents

1	Introduction	3
1.1	Problem Definition, Strategy and Metrics	3
2	Data Exploration and Preprocessing	3
2.1	Dataset: Nutrition5K	3
2.2	Data Cleaning and Exploration	3
2.2.1	Filtering Outliers	4
2.3	Data Augmentation via Frame Extraction	6
3	Segmentation Analysis	8
3.1	Motivation and Problem Formulation	8
3.2	U ² -Net	9
3.3	Implementation and Training Details	9
3.4	Segmentation Pipeline and Evaluation Methodology	9
3.4.1	Experimental Setup	10
3.4.2	Testing protocol	10
3.5	Results and Discussion	10
3.5.1	Impact on Final Architecture	11
4	Dish Weight Estimation	12
4.1	Model Architectures	12
4.2	Performance Evaluation	12
5	Relative Nutrient Estimation (per 100g)	13
5.1	Comparative Model Analysis	13
5.1.1	SimpleCNN	15
5.1.2	DeepCNN	15
5.1.3	MobileLikeNet	15
5.1.4	ResNet34	15
5.2	Percentage estimation results	15
6	Combined Model and Final Results	16
7	Conclusion and Future Work	16

1 Introduction

1.1 Problem Definition, Strategy and Metrics

With growing attention brought to ML and latest improvements in the field, more and more problems that seemed unresolvable before now can be solved. One of these tasks is estimating nutrition of the meal by taking a picture of it. This is a highly researched topic and demand for solution is insanely high. In general the problem is quite sophisticated, because even the companies themselves are allowed to have 20% marginal error for calories estimation of their products. Many datasets, like *Open Food Facts* [2] exploit these errors, except *Nutrition5k* [5] which seems to resolve this issue by manual labeling with professional nutritionists.

We made an attempt to solve the limited task as well of trying to predict calories, fats, carbs and proteins by a picture of the meal. One of our inspirations of the methodology [5] is that we will not predict absolute values, but split the task in two - predicting the weight of the model and predicting the relative values per 100g. We employed standard regression metrics to assess segmentation impact:

- **Mean Absolute Error (MAE):** $MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$
- **Relative MAE:** $MAE_{\%} = \frac{MAE}{\bar{y}} \times 100\%$
- **Root Mean Square Error (RMSE):** $RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$
- **Coefficient of Determination:** $R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$

2 Data Exploration and Preprocessing

2.1 Dataset: Nutrition5K

The Nutrition5K dataset [5] is a comprehensive collection of realistic dishes from Google cafeterias. It was released as part of the Google Research CVPR 2021 paper [5] on vision-based nutrition understanding, making it suitable for our intended goals. In addition, the availability of a high-quality benchmark paper constitutes an interesting point of comparison for us.

The dataset comprises 5,006 dish scans of varying nutritional contents. Each dish scan contains four rotating side-angle videos, a detailed list of ingredients with corresponding per-ingredient and per-macronutrient (calories, weight, protein, fat and carbohydrates) masses, as well as overhead RGB-D images for approximately two-thirds of the samples. The latter consist of RGB images with additional colorized depth images taken with an Intel RealSense camera. We do not make use of the depth images, and only use the RGB images for our models. This choice is motivated by the end goal of the project : being able to determine nutritional contents of a dish using exclusively a single picture from a smartphone.

The dataset is structured as follows: each image is located in `imagery/realsense_overhead/dish_[ID]/`, where `dish_[ID]` is a unique identifier assigned to every dish. This folder contains RGB (`rgb.png`), raw depth (`depth_raw.png`), and colorized depth images (`depth_color.png`). The side-angle videos are located in `imagery/side_angles/dish_[ID]/` with four video files, each capturing the dish from different angles. This proved useful when we explored one of the data augmentation techniques in (subsection 2.3).

The nutritional ground truth is provided in CSV format across two café locations (under `metadata/`, with `dish_metadata_cafe1.csv` and `dish_metadata_cafe2.csv`), and the train/test split used by Google Research to compute the published performance metrics is available under `dish_ids/splits/`.

2.2 Data Cleaning and Exploration

Data preprocessing plays a key role in ensuring reliable nutrition prediction and helps models generalize more effectively. We built a data cleaning pipeline that addresses several issues we encountered in the raw Nutrition5k dataset.

2.2.1 Filtering Outliers

We start building the dataset to be used throughout the entire pipeline by an error-correcting CSV parsing that handles malformed entries, and only selecting the desired targets.

```
dishes.append({
    "dish_id": dish_id,
    "calories": float(parts[1]),
    "weight": float(parts[2]),
    "fat": float(parts[3]),
    "carbs": float(parts[4]),
    "protein": float(parts[5]),
})
```

We then implement a filtering stage that removes outliers (pathological data points with physiologically implausible nutritional facts). Our analysis revealed a significant number of outliers, especially dishes with < 5 or > 2000 calories (considering that the dishes in the dataset mostly contain whole, minimally processed foods).

We start by the following:

- We filter out all dishes without an overhead image. Even when side-angle videos are available, we keep only the data points with overhead images, as our end goal is to process dish photos taken with a smartphone, which are usually taken from slightly elevated angles, most similar to the overhead images.
- For EDA, we impute using kNN ($k=5$) on standardized targets among {calories, weight, fat, carbs, protein}. However, we never use imputed labels for training, as missingness is unlikely to be MCAR (Missing Completely At Random) and may bias estimates.

However, as shown in Figure 1a, the raw data still contains extreme tails in macronutrient composition that can destabilize training if left unchecked. To mitigate their influence while preserving legitimately large meals, we use the following:

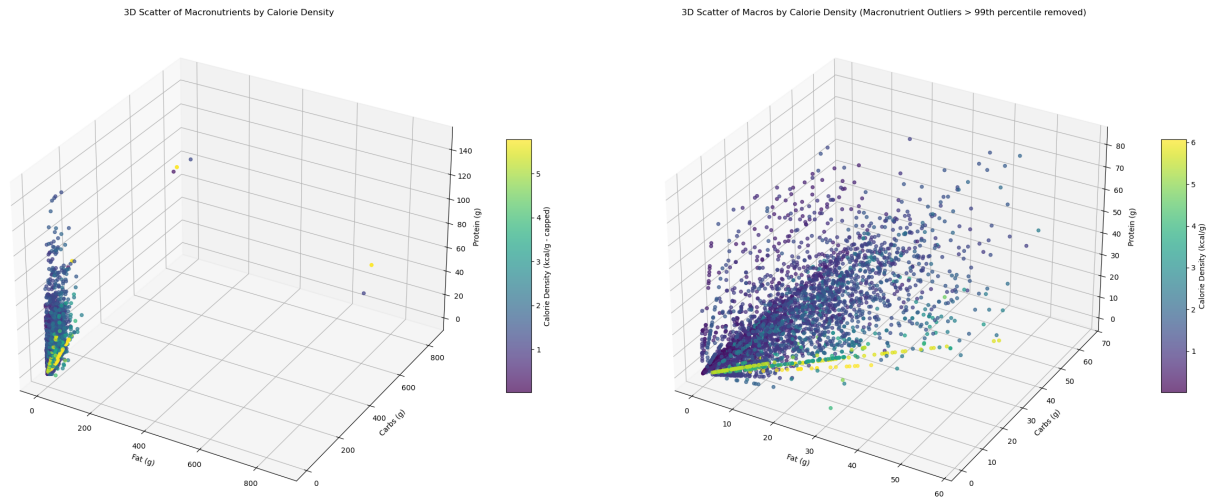
- We normalize macronutrients by dish weight (per 100 g) and cap per-100 g values at the 1st and 99th percentiles. This is computed on the training split only to avoid test leakage. This reduces the effect of heavy tails.
- We drop only samples that clearly violate nutritional plausibility: null and negative targets, calories-per-gram > 9.5 kcal/g or sum of macros exceeding dish weight by more than 10%.

A visualization of the underlying data structure is now clearer, as seen in Figure 1b: High-density dishes (yellow) typically have a larger fat composition (9 kcal/g), clustering closer to the fat axis, while dishes with lower caloric density (blue) spread toward carb and protein regions, further from the fat axis.

As a final sanity check, we compute the Pearson correlation matrix for the five target variables to assess dataset nutritional consistency. See Figure 2.

Fat demonstrates the highest correlation with calories (0.92), significantly higher than carbohydrates (0.68) and protein (0.62), which aligns with their respective energy densities, while dish weight shows expected moderate positive correlation.

Thus, the cleaned dataset is a reliable foundation for training a multi-target nutritional prediction model.



(a) Macronutrient distribution before outlier removal. Note the stretched axes due to the extreme points.

(b) Macronutrient distribution after per-100 g percentile capping and plausibility cleaning.

Figure 1: 3D scatter plot of macronutrients (fat, carbs, protein) colored by calorie density, shown before and after cleaning. The data cleaning process makes the central cluster of common dishes items more apparent.

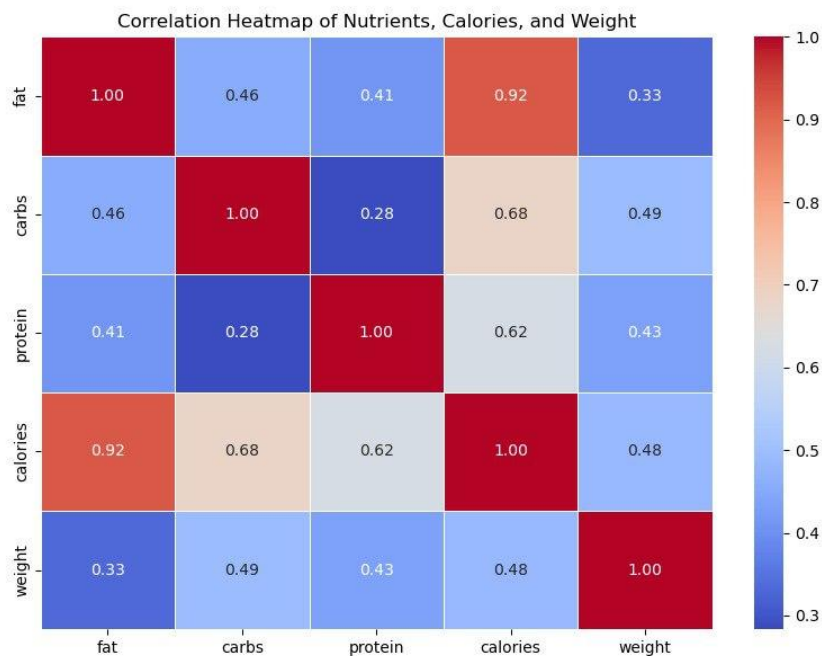


Figure 2: Correlation heatmap of the target nutritional variables after data cleaning.

2.3 Data Augmentation via Frame Extraction

To improve model generalization, we decided to take advantage of videos by extracting frames from them and using them as distinct data entries. However, if we take every frame for our dataset we will end up with terabytes of data which is unmanageable within our capacities. Therefore, we need to understand how to extract small enough amount of frames to preserve lightning variability and viewpoint diversity.

One way to distinguish them is performing Principal Component Analysis (PCA) and select frames from different clusters. We utilize the Gaussian Mixture Model (GMM) to find the best clustering. As a criterion we test for $k = 1 \dots 10$ clusters to predict and calculate the Bayesian Information Criterion (BIC). BIC would give us the best number of clusters that we should pick per video. The best BIC score is achieved for 8 clusters per

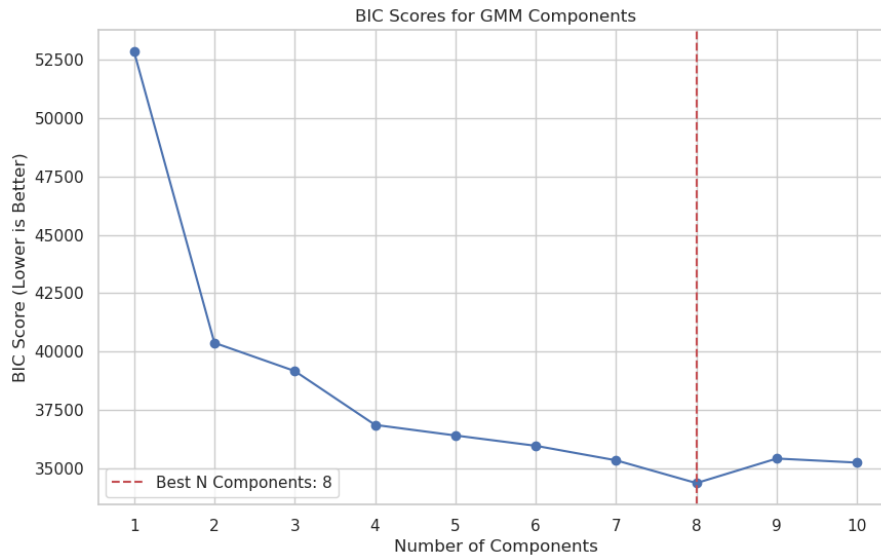
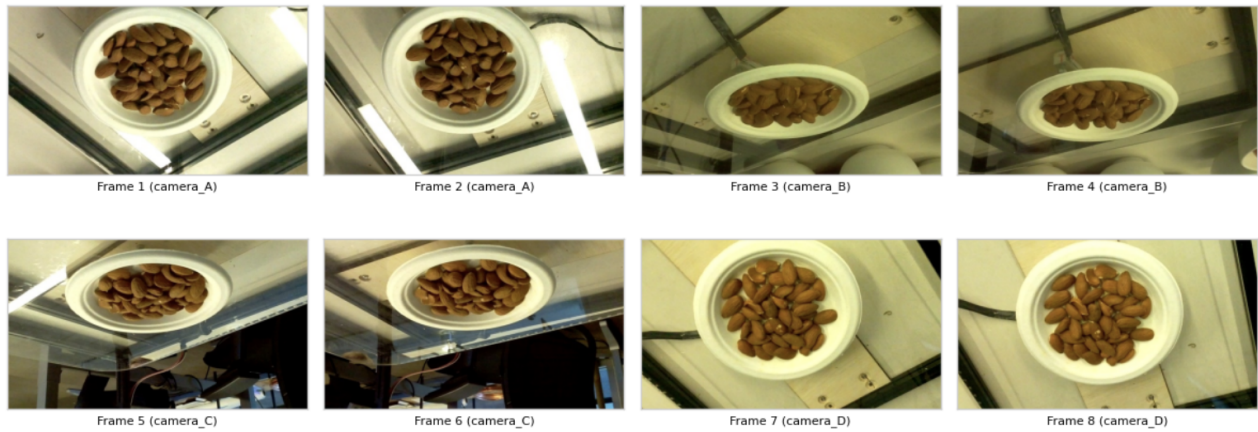


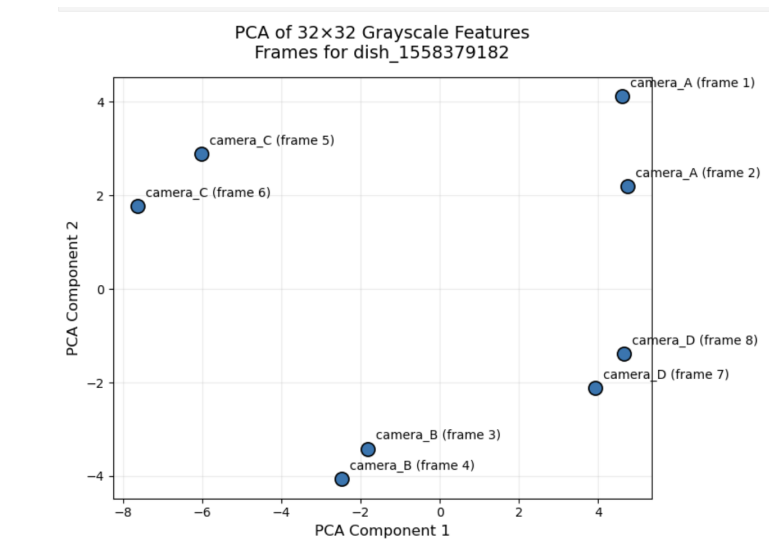
Figure 3: BIC Scores for different clusters 1..10. *It is important to say, that trying to fit for more clusters (more than 10) may get better results, but we don't have the capacity either way to take so many frames in the first place.*

video. However, taking 8 frames per video would result in approximately 160 GB of training data, which is not feasible considering our limitations. Therefore, using the elbow rule we can safely take 2 frames, which expand our dataset by 8 times. Even though the frames form clusters that are not necessarily independent, they are still sufficiently diverse to be considered as training data entries.

The example in Figure 4 illustrates how the process works for a single dish. We can see that the extracted frames, when projected using PCA, form 4 visually distinct clusters corresponding to different camera angles. Although each cluster contains 2 images, it is not a problem, since we are applying the same rule uniformly across all dishes. Moreover, this allows to capture viewpoint and lighting variability. Therefore, our data augmentation approach is consistent, and it is used throughout the remainder of the pipeline.



(a) Extracted frames from one dish video



(b) PCA of grayscale frame embeddings

Figure 4: Examples of extracted frames and their PCA projection. Frame diversity captures changes in viewpoint and lighting, forming distinct visual clusters.

3 Segmentation Analysis

3.1 Motivation and Problem Formulation

During early testing, our CNN regressors exhibited instability when trained on raw images with cluttered backgrounds and variable lighting. Small portions occupying a tiny fraction of the plate were particularly problematic. This motivated us to investigate the addition of a segmentation preprocessing stage.

The following failure modes were identified after visual inspection of predictions on similar dishes, where the only main differences were the additional presence of items in the background (1) and/or shadow artifacts (2):

1. Models seem to incorporate non-food elements including plates and utensils into nutritional predictions, leading to systematic overestimation.
2. Lighting variations and shadows caused inconsistent predictions for identical food items, often leading to underestimation.

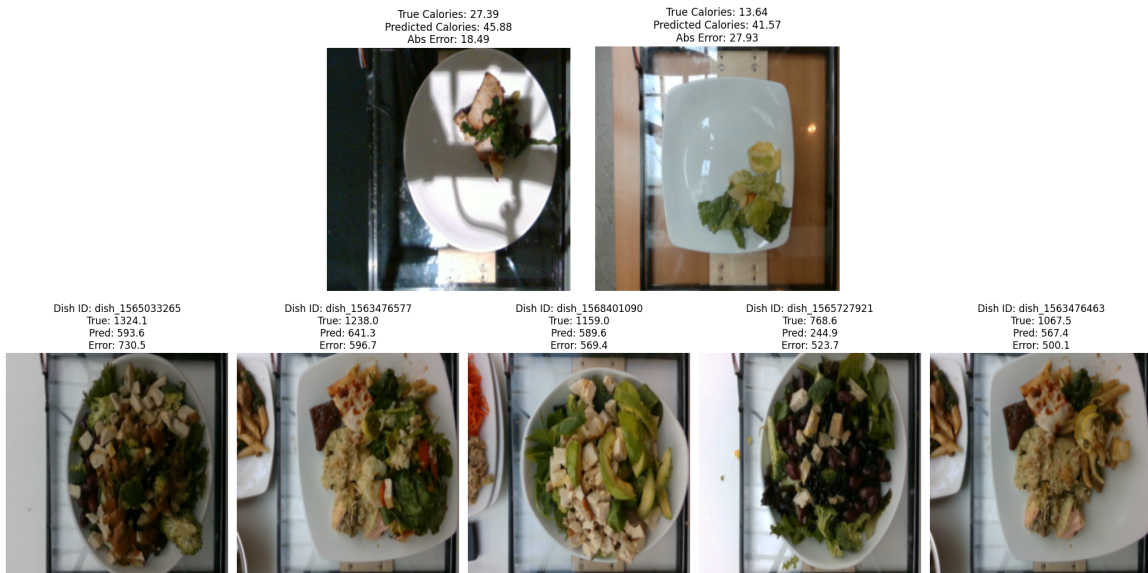


Figure 5: Examples of failure cases that led to this segmentation experiment.

To address these challenges, we experiment with a segmentation preprocessing stage, with the goal of focusing the model on nutritionally relevant regions. This is done during both training and inference to evaluate the impact of this preprocessing according to the metrics defined in subsection 1.1.

3.2 U²-Net

After testing different YOLO [4] models, as well as SAM [1] models, we chose U²-Net [3] as our primary segmentation backbone as it demonstrates good performance in salient object detection. Most importantly, the nested U²-Net architecture is capable of extracting features at multiple scales, which is essential as dishes often present diverse textures/colors/ingredients that require different levels of abstraction for accurate segmentation.

3.3 Implementation and Training Details

Nutrition5K does not include ground-truth segmentation masks. Since our goal is not to train a segmentation model, we adopt a saliency-based strategy centered on a publicly available U²-Net checkpoint [3] trained for general salient object detection. We decide not to fine-tune nor train from scratch on food segmentation datasets because of the time and computational constraints of the project.

Input preprocessing We adopt a 64/16/20 train/val/test split. Images are resized to 224×224 and normalized using ImageNet statistics (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]) to match the expected output from the used U²-Net checkpoint. We apply simple augmentations (random horizontal flips, random rotations and small color jitter) to encourage robustness to illumination changes as well as generalization to different viewpoints.

Loss function Segmentation being used purely as a preprocessing stage, the regression CNN is trained with mean squared error (MSE) on the target variables:

$$\hat{y}_i = f_\theta(x_i) \in \mathbb{R}^D, \quad y_i \in \mathbb{R}^D, \quad (1)$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{ND} \sum_{i=1}^N \sum_{d=1}^D (\hat{y}_{i,d} - y_{i,d})^2. \quad (2)$$

where x_i denotes the preprocessed RGB image fed to the regressor (either raw or segmented; tensor of shape $3 \times 224 \times 224$), f_θ is the ResNet50-based regression network with learnable parameters θ , $\hat{y}_i \in \mathbb{R}^D$ is the predicted vector (e.g., calories only $D=1$; calories+weight $D=2$; all targets such as calories+protein+carbs+fat $D=4$), $y_i \in \mathbb{R}^D$ is the ground-truth vector, D is the number of target variables, i indexes samples in a mini-batch, and N is the number of samples in the current mini-batch.

Optimization and regularization We use AdamW (initial learning rate 1×10^{-3} , weight decay 1×10^{-4}) together with ReduceLROnPlateau on the validation loss (factor 0.2, patience equal to half of the early-stopping patience; 5). Early stopping monitors validation loss with patience 10, and the best model checkpoint is saved whenever the monitored loss improves.

3.4 Segmentation Pipeline and Evaluation Methodology

The complete segmentation pipeline takes raw food images and outputs isolated food regions (background is composited to black) through the following process:

For any given dish image, the pipeline checks if a segmentation mask has already been computed and cached, in which case it uses it. Otherwise, the U²-Net model is used to compute a grayscale saliency map which is then thresholded at $\tau=0.5$ into a binary mask. The latter is applied to the original image to highlight the main food items. The resulting image is then cropped to an axis-aligned bounding box of the largest contour (largest connected component, supposed to be the food region on the dish).

Architectures evaluated in segmentation experiments We paired the preprocessing conditions with the following ResNet50-based regressor:

- **ResNet50-based regressor:** ResNet50 backbone (frozen) with an MLP head.

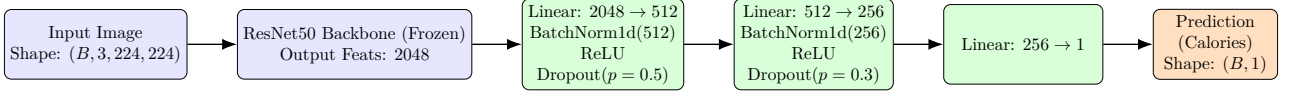


Figure 6: ResNet50-based regressor. **Total parameters:** $\sim 24.6\text{M}$ (*frozen backbone:* $\sim 23.5\text{M}$), **trainable parameters:** $\sim 1.18\text{M}$

3.4.1 Experimental Setup

To quantify the impact of this segmentation pipeline and to assess its relevance as a preprocessing step, we conducted controlled experiments and compared three image preprocessing approaches in total:

1. Original RGB images without preprocessing
2. Simple geometric square cropping based at the center of the images
3. Full segmentation pipeline with blacked-out background as described above.

3.4.2 Testing protocol

To isolate the effect of the segmentation preprocessing, we hold relevant RNG seeds, the regressor head architecture, optimizer hyperparameters, and training schedule fixed across all tests.

We defined a protocol to assess the performance impact of segmentation vs. no segmentation. The following configurations were evaluated:

Table 1: Segmentation testing protocol matrix

Model	Target	Dataset	Segmentation
ResNet50 regressor	Calories	Overhead only	No / Yes
ResNet50 regressor	Calories	Overhead + Extracted (PCA+GMM)	No / Yes
ResNet50 regressor	All nutrients	Overhead only	No / Yes

3.5 Results and Discussion

Contrary to our initial expectations, the segmentation preprocessing step does not improve nutritional prediction accuracy. Possible suspects include the loss of information (contextual cues such as plate scale) and undesired artifacts from the segmentation process on some challenging samples, as can be seen in Figure 7.

We also conduct a qualitative analysis of some predictions with the segmentation step through visual inspection. The pre-trained U²-Net segmentation model performs particularly well on dishes with high contrast between food and background, instances in which the segmentation masks successfully exclude unrelated elements from the image such as cutlery or tableware. We however also observe both over-segmentation of dissimilar food items (e.g., separating sauce from base ingredients) and segmentation failures on some challenging samples (e.g., food lies in the blacked-out area).



Figure 7: U²-Net segmentation performance across different food types

Table 2: Comparative Prediction Performance: Without vs. With Segmentation

Nutrient	Metric	Without Seg.	With Seg.	Change (%)
Calories	MAE	81.27	90.36	+11.18% ↑
	MAE (%)	31.91	35.48	+11.18% ↑
	RMSE	116.98	130.89	+11.89% ↑
	R²	0.68	0.60	-11.76% ↓
Weight	MAE	46.09	64.87	+40.75% ↑
	MAE (%)	20.79	29.27	+40.79% ↑
	RMSE	65.05	89.59	+37.72% ↑
	R²	0.82	0.66	-19.51% ↓
Fat	MAE	7.25	6.45	-11.03% ↓
	MAE (%)	58.20	51.81	-10.98% ↓
	RMSE	10.27	9.95	-3.12% ↓
	R²	0.43	0.46	+6.98% ↑
Carbs	MAE	9.54	10.44	+9.43% ↑
	MAE (%)	49.40	54.04	+9.40% ↑
	RMSE	12.98	14.05	+8.24% ↑
	R²	0.34	0.23	-32.35% ↓
Protein	MAE	9.59	10.12	+5.53% ↑
	MAE (%)	50.77	53.54	+5.46% ↑
	RMSE	14.22	15.29	+7.52% ↑
	R²	0.55	0.48	-12.73% ↓

Note: **Bold** values indicate the best performance. The “Change (%)” column shows the performance shift from the “Without Seg” model to the “With Seg” model. *Green indicates an improvement* (lower error, higher R²), while *red indicates a decline*.

3.5.1 Impact on Final Architecture

Given the observed performance degradations (except a small improvement for fat), we remove segmentation in the final model and retain the simpler center cropping step that preserves contextual cues and reduces compute.

4 Dish Weight Estimation

This part of the project focuses on specifically estimating the total weight of a dish from a single image. We took this approach because we believe that accurate weight estimation is fundamental in determining all the nutritional components. On top of that, it allows the conversion of relative nutrition estimations (from our %-estimation models) into numerical values, which we do later in the pipeline.

We explore two different model architectures (built from scratch) of increasing complexity and test them on both the original Nutrition 5K dataset and the augmented one (see subsection 2.3). Our results demonstrate that deeper architectures combined with the augmented dataset lead to better performance, achieving a relative MAE of 18.68% on the testing dataset.

4.1 Model Architectures

We design and implement two CNN architectures with varying complexity to understand the relationship between model capacity and weight estimation performance.

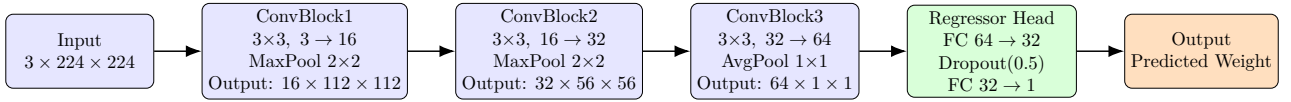


Figure 8: SimpleCNN architecture for weight estimation (25k parameters)

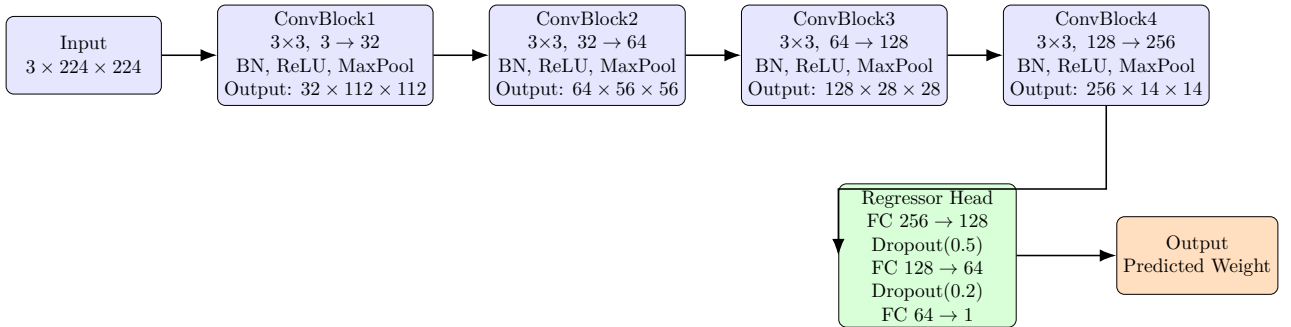


Figure 9: DeepCNN architecture for weight estimation (430k parameters)

Additionally, we explore the benefit of data augmentation by training on an extended dataset where extra frames are extracted from side-angle dish videos. This augmentation increases data diversity in terms of lighting and viewpoint.

4.2 Performance Evaluation

The performance is assessed using the metrics fixed in the beginning.

Table 3: Dish-weight prediction results on the Nutrition5K test split. “Extended” denotes training with the two-frame augmented set.

Model	MAE	Rel. MAE (%)	RMSE	R ²	Mean True	Mean Pred
Deep-CNN (extended)	43.89	18.68	63.12	0.82	234.9	229.2
Deep-CNN	43.54	21.31	57.82	0.84	204.3	208.0
Simple-CNN	76.11	37.26	103.91	0.49	204.3	202.3

Baseline performance: On the original Nutrion5K dataset, the DeepCNN model performed noticeably better than the SimpleCNN model, lowering the MAE by 43% and raising R^2 from 0.49 to 0.84. At the same time, the relative MAE was improved significantly, going from 37.26% to 21.31%. These findings suggest that the more complex architecture more effectively captured the visual characteristics required for accurate weight estimation

Impact of Data Augmentation: Using the augmented dataset resulted in further performance improvement. The R^2 remained roughly constant at 0.82, while the DeepCNN model generated a lower relative MAE, falling to 18.68%. These results validated the data augmentation strategy and confirmed that combining the more complex architecture with the augmented dataset resulted in the best performance.

Later in the pipeline, we proceeded with the DeepCNN model trained on the augmented dataset for the weight estimation part.

5 Relative Nutrient Estimation (per 100g)

5.1 Comparative Model Analysis

The approach we chose is different to predicting absolute values. We train two models separately - one predicting the weight of the food on the picture and another one predicting how many nutrition per 100g in this meal. This result for a picture is later multiplied by the weight predicted, to achieve the total nutrition per meal. We were inspired by the paper model with well-researched architecture and tried to enhance it to suit our limitations and resources. We compared the four categories of models.

- **Simple CNN:** Simple Convolutional Network to better understand the task and set a baseline fundamental performance.
- **DeepCNN:** A deeper custom architecture, designed to test the performance limits of a network trained from scratch.
- **MobileLikeNet:** A lightweight computationally efficient model designed for applied user-case of deployment on mobile phones.
- **ResNet34:** A deep, pre-trained model utilizing transfer learning, aiming for best performance.

- starting from SimpleCNN exploratory model for eased understanding of the problem to ResNet34 - heavy pre-trained model to achieve best theoretical results.

All models were trained with a batch size of 32 for 200 epochs, using 80% of the data per epoch and an early stopping patience of 10.

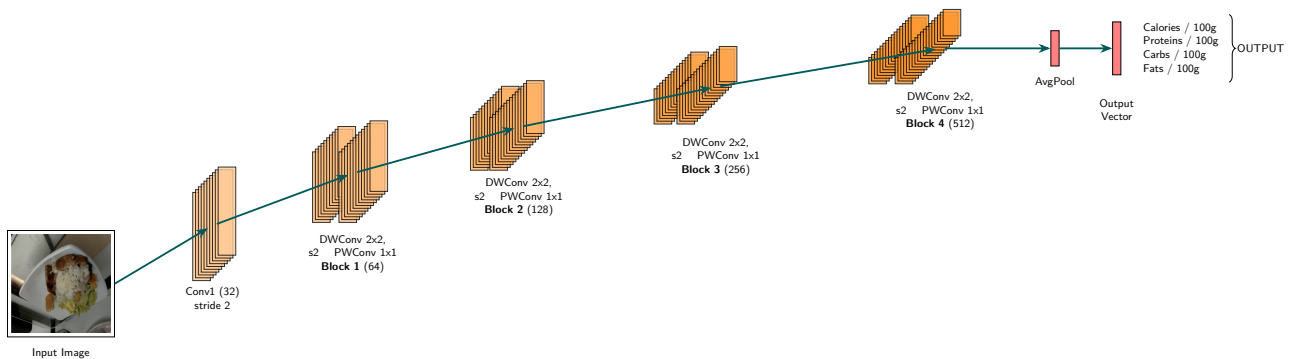


Figure 10: Mobile-Like Net

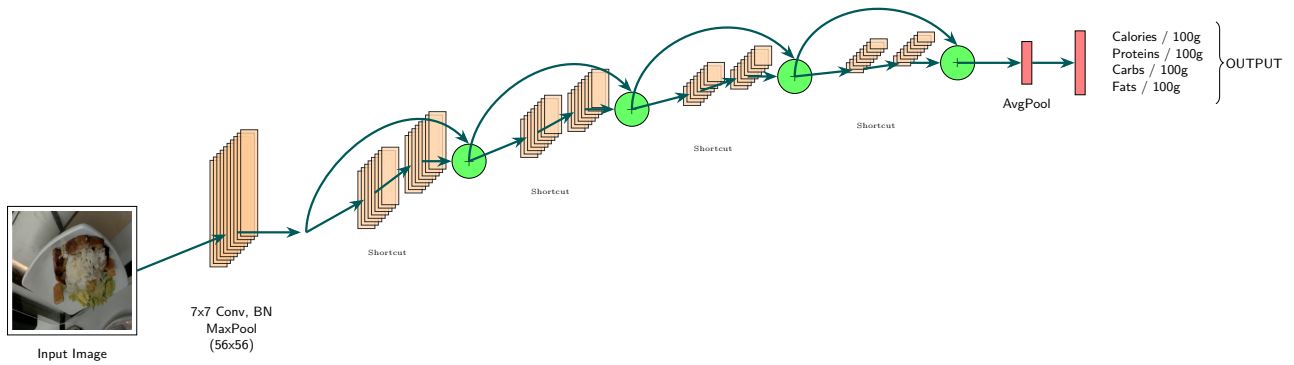


Figure 11: Deep CNN

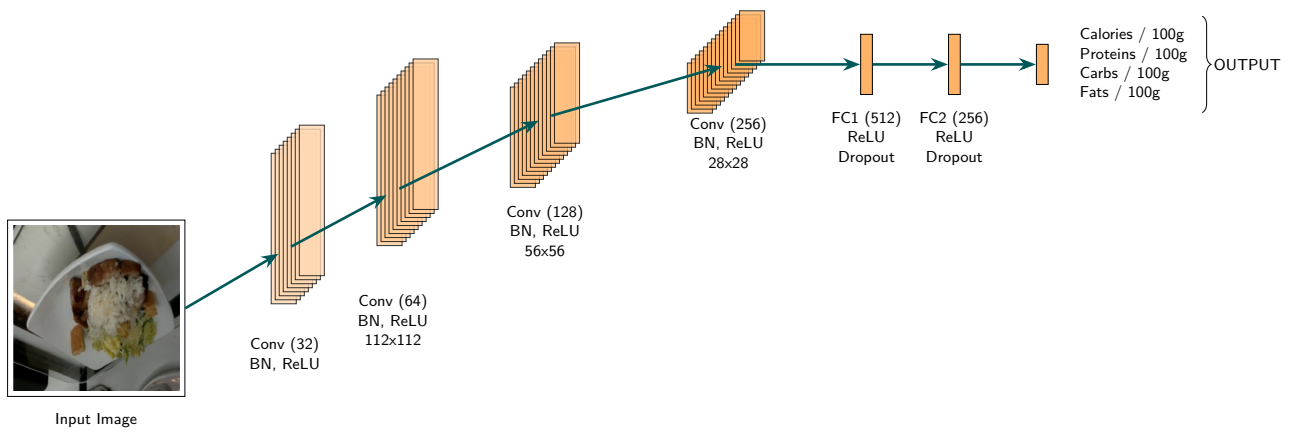


Figure 12: SimpleCNN

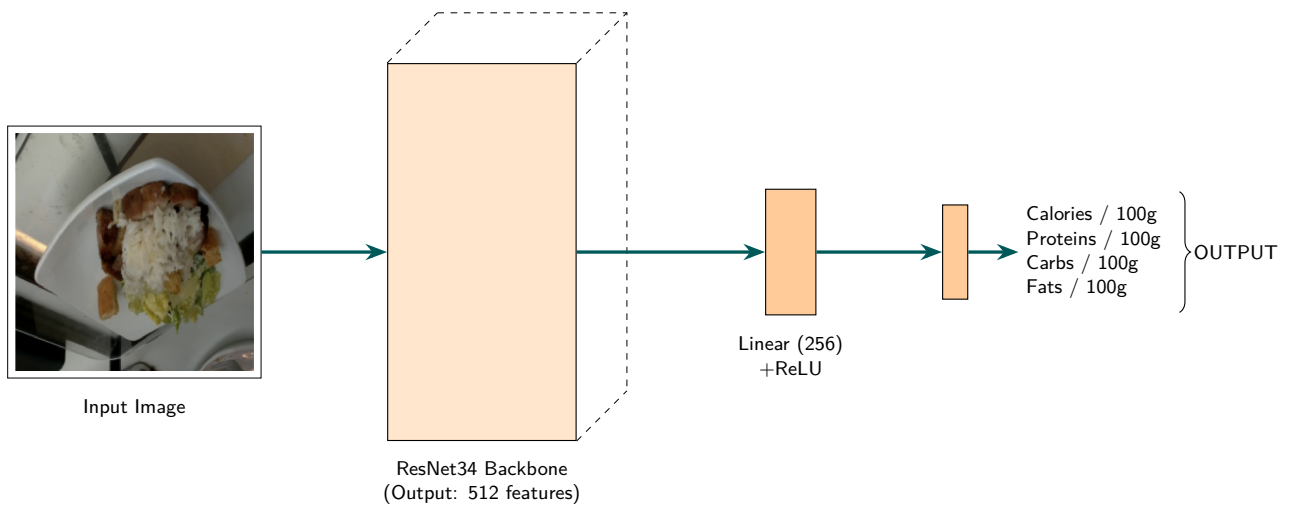


Figure 13: A pre-trained ResNet34 backbone

5.1.1 SimpleCNN

For the purpose of better understanding the problem and simplicity we start with so-called SimpleCNN which is essentially a 4-layer Convolution Network.

Figure 12 serves as our baseline. ($LR: 1e-3$, $Params: 26.2M$).

5.1.2 DeepCNN

To push our capabilities to the maximum we explore DeepCNN model - Deep Convolution Neural Network to try to achieve the best results without pre-trained models.

9 Convolution layers, followed by average pooling layer and fully-connected layer (Figure 11) .($LR: 1e-4$, $Params: 9.8M$). Surprisingly, the results seem to be the same as for Simple CNN, maybe because of the early stopping we couldn't fully exploit deep cnn abilities.

5.1.3 MobileLikeNet

Our third explored model is user-oriented, since calories estimation is prominent topic and demand for accurate solutions is very high, we explore applied models that can be installed on phones and mobile devices. One such model would be MobileLike Network - light CNN that is not computationally heavy. ($LR: 1e-4$, $Params: 184k$).

5.1.4 ResNet34

(Figure 13) is a standard, very deep architecture pre-trained on ImageNet. We fine-tuned it for our regression task. ($LR: 1e-5$, $Params: 21.4M$).

5.2 Percentage estimation results

Table 4: Performance Metrics of Different CNN Models for Nutrient Estimation. Best values are in **bold**.

Model	Nutrient	Extended dataset				Original dataset			
		MAE	RMSE	R ²	MAE %	MAE	RMSE	R ²	MAE %
Simple CNN	calories.per_100g	41.068	60.153	0.626	26.881	47.272	70.231	0.608	29.086
	carbs.per_100g	6.097	9.163	0.011	55.442	5.587	8.138	0.102	52.258
	fat.per_100g	3.654	5.417	0.593	42.646	3.881	5.812	0.652	40.347
	protein.per_100g	4.160	5.420	0.239	43.975	4.081	5.397	0.305	40.861
DeepCNN	calories.per_100g	39.945	58.060	0.651	26.146	41.080	58.477	0.729	25.276
	carbs.per_100g	4.596	7.004	0.422	41.793	5.373	7.692	0.198	50.253
	fat.per_100g	3.449	5.040	0.648	40.249	3.351	5.043	0.738	34.843
	protein.per_100g	3.571	4.742	0.417	37.747	3.663	4.959	0.414	36.673
Mobile-like Net	calories.per_100g	49.930	74.671	0.423	32.681	53.563	80.529	0.485	32.957
	carbs.per_100g	5.616	8.645	0.119	51.067	5.553	8.319	0.062	51.942
	fat.per_100g	4.286	6.552	0.405	50.016	4.468	6.919	0.507	46.450
	protein.per_100g	4.114	5.372	0.252	43.494	3.964	5.243	0.345	39.685
ResNet34	calories.per_100g	22.899	38.056	0.850	14.988	26.829	38.928	0.880	16.507
	carbs.per_100g	2.683	4.254	0.787	24.396	3.419	5.126	0.644	31.984
	fat.per_100g	2.130	3.421	0.838	24.854	2.794	3.981	0.837	29.043
	protein.per_100g	2.175	3.125	0.747	22.997	2.959	3.834	0.649	29.627

Performance analysis *ResNet34* achieved the best results across all metrics, which is not surprising, since it is the most computationally heavy out of all and additionally, pre-trained for a similar task. On the full (extended) dataset, it achieves only **22.8 MAE** with 14.98% Error, which is extremely accurate (for comparison professional nutritionist predicts with 50% accuracy).

The **DeepCNN** and **Simple CNN** models delivered comparable, moderate results. Small difference in their performance suggests, that there was not enough data to fully exploit DeepCNN features, even with residual blocks.

6 Combined Model and Final Results

After training each part, we combine our best weight estimation model (DeepCNN) with our best relative nutrient estimation (ResNet34) to produce absolute predictions by multiplying the output of the 2 models. We summarize the results into a comprehensible table below, verifying on whole dataset.

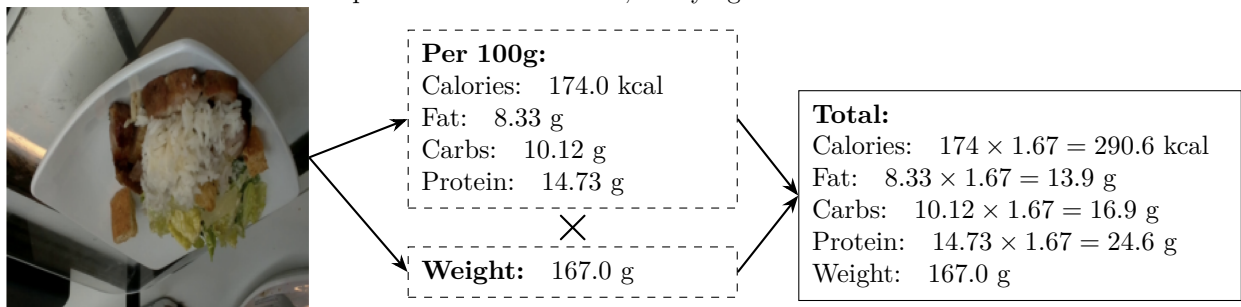


Table 5: Performance metrics of combined model: Weight est. * Per100g est.

Nutrient	MAE	RMSE	R ²	MAE %	Mean True	Mean Pred
Calories (abs)	87.495	128.939	0.607	28.552	306.448	304.311
Fat (abs)	6.045	8.975	0.519	37.583	16.084	15.858
Carbs (abs)	7.760	12.154	0.449	37.008	20.969	21.700
Protein (abs)	7.588	11.938	0.686	34.080	22.265	20.851
Weight (g)	49.084	74.165	0.758	21.339	230.023	226.192

These results confirm that combining the two models together is indeed a reasonable approach. The pipeline demonstrates stable performance on the full dataset, predicting calories with a 28.552 MAE %. These results set a solid foundation for further research and development on this topic, which we discuss in the next chapter.

7 Conclusion and Future Work

Overall, the model we created is a robust approach for the challenging problem, achieving quite good results. Furthermore, this model can be trained on different cuisines and produce more consistent estimations than a human expert in some cases, who can have limited knowledge on rarer types of dishes.

Another interesting point to consider is the cost of access to a professional nutritionist. For some people, nutrition is not necessarily a priority or might not even be considered an important health factor due to poor education or simple negligence. This is where solutions like FoodCNN might make a difference by allowing easy

access to cheap and fast nutrition estimation to get insights into one's eating habits and how they could be improved upon.

Some notable limitations we encountered and that could be improved upon in a future version of this project:

- Deep learning requires significant resources in terms of time and computational power that need to be taken into account.
- Segmentation could likely be improved by training a dedicated segmentation model on food data, which was not done due to time and complexity constraints.
- The Nutrition5k dataset was relatively small for the given task, and contained many small dishes. More food datasets could be used to allow the model to train on more examples.

References

- [1] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alex Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. 2023.
- [2] Alexandre Lemercier. Food - detailed nutritional content. <https://www.kaggle.com/datasets/alexandrelemercier/food-detailed-nutritional-content>, 2021. Kaggle dataset.
- [3] Xuebin Qin, Zichen Zhang, Chenyang Huang, Masood Dehghan, Osmar R. Zaiane, and Martin Jagersand. U²-net: Going deeper with nested u-structure for salient object detection. 2020. arXiv:2005.09007 [cs.CV].
- [4] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. 2016.
- [5] Quin Thames, Arjun Karpur, Wade Norris, Fangting Xia, Liviu Panait, Tobias Weyand, and Jack Sim. Nutrition5k: Towards automatic nutritional understanding of generic food. *arXiv preprint arXiv:2103.03375*, 2021.